



Aufgabe 1 (10 Punkte)

Diese Aufgabe beschäftigt sich damit, dass die Armijo-Schrittweitenbestimmung für gegebene Suchrichtungen p^k unzureichende Ergebnisse liefern kann. Wir betrachten dazu die Funktion $f(x) := x^2/8$ und ein iteratives Verfahren $x^{k+1} = x^k + \alpha_k p^k$ mit Startwert $x^0 > 0$ und Suchrichtungen $p^k := -f'(x^k)/2^k$. Die Schrittweiten α_k werden jeweils mit der Backtracking Line Search für die Armijo-Bedingung (Algorithmus 2.5.6) beginnend bei $\alpha = 1$ bestimmt.

- a) Zeigen Sie, dass die Armijo-Bedingung mit Parameter $c_1 \leq 1/2$ in jedem Schritt k der Iteration für die Schrittweitenwahl $\alpha_k = 1$ erfüllt ist.
- b) Zeigen Sie, dass dieses Verfahren gegen ein $\tilde{x} \geq x^0/2$ (also insbesondere nicht gegen den Minimierer $x^* = 0$) konvergiert.

Hinweis: Zeigen Sie zuerst induktiv $0 < x^{k+1} < x^k$ für alle $k \geq 0$ und dann $x^0 - x^k \leq x^0/2$ für alle $k \geq 1$.

- c) Sei $c_2 \in (0, 1)$ unabhängig von k gewählt. Zeigen Sie, dass die Schrittweite $\alpha_k = 1$ für k groß genug die *Krümmungsbedingung*

$$f'(x^k + \alpha_k p^k) p^k \geq c_2 f'(x^k) p^k$$

nicht erfüllt.

- d) Warum wird die Armijo-Bedingung $f(\mathbf{x}^k + \alpha \mathbf{p}^k) \leq f(\mathbf{x}^k) + c_1 \alpha \nabla f(\mathbf{x}^k)^\top \mathbf{p}^k$ *sufficient decrease-Bedingung* genannt?
- e) Warum wird die Bedingung $\nabla f(\mathbf{x}^k + \alpha \mathbf{p}^k)^\top \mathbf{p}^k \geq c_2 \nabla f(\mathbf{x}^k)^\top \mathbf{p}^k$ *Krümmungsbedingung* genannt?

Aufgabe 2 (5 Punkte)

- a) Wir betrachten das Gradientenabstiegsverfahren für $f : \mathbb{R} \rightarrow \mathbb{R}$, $f(x) = \frac{1}{2}x^2$ mit $x^0 = 1$. Zeigen Sie, dass in der ersten Iteration keine Schrittweite $\alpha_0 > 0$ existiert, für die die Wolfe-Bedingungen mit Parametern $c_1 = 0.7$, $c_2 = 0.3$ erfüllt sind.

Wieso stellt dies keinen Widerspruch zu Satz 2.5.3 dar, der uns die Existenz einer solchen Schrittweite garantieren sollte?

- b) Gegeben sei ein Quasi-Newton-Verfahren, d.h. $x^{k+1} = x^k + \alpha_k p^k$ mit $p_k = -B_k \nabla f(x^k)$, wobei für alle $k \in \mathbb{N}_0$ die Matrix B_k symmetrisch positiv definit und von gleichmäßig beschränkter Kondition sei, d.h. es existiert $M > 0$ sodass

$$\kappa(B_k) := \frac{\lambda_{\max}(B_k)}{\lambda_{\min}(B_k)} \leq M \quad \forall k \in \mathbb{N}_0.$$

Zeigen Sie, dass für das Verfahren unter den Voraussetzungen von Satz 2.5.4 (*Zoutendijk*) gilt, dass

$$\lim_{k \rightarrow \infty} \left\| \nabla f(x^k) \right\|_2 = 0.$$

Hinweis: Hilfreich könnte folgende Aussage sein, die bereits auf Blatt 5 bewiesen wurde: Für eine spd. Matrix $A \in \mathbb{R}^{n \times n}$ mit minimalem und maximalem Eigenwert $\lambda_{\min}$ bzw. $\lambda_{\max}$ und bel. $x \in \mathbb{R}^n$ gilt

$$\lambda_{\min} \|x\|_2^2 \leq x^\top A x \leq \lambda_{\max} \|x\|_2^2.$$

Aufgabe 3 (15 Punkte)

- a) Auf dem letzten Übungsblatt wurde das ungedämpfte Newtonverfahren implementiert. Dieses soll nun – ebenso wie das Gradientenabstiegsverfahren aus Übung 5 – dahingehend erweitert werden, dass die Wahl der Schrittweite mittels Armijo-Backtracking-Liniensuche (Algorithmus 2.5.6 im Skript) erfolgt. Zudem soll die Optimierung nun anhand der Abbruchkriterien nach Gill, Murray und Wright (Bemerkung 2.5.8 im Skript) beendet werden.

- (i) Implementieren Sie für die Armijo-Backtracking-Liniensuche die Funktion

```
def armijo(f, p, x, rho, c1, alpha0=1):  
    ...  
    return alpha
```

die für die Parameter `rho`, `c1` und zu gegebener Funktion `f` mit Abstiegsrichtung `p` und Iterationspunkt `x` die berechnete Schrittweite `alpha` zurückgibt. Die Liniensuche soll dabei mit `alpha0` initialisiert werden. Geben Sie in der Konsole aus, falls nach 100 Iterationen keine Schrittweite gefunden wurde, die die Armijo-Bedingung erfüllt.

- (ii) Implementieren Sie die Funktion

```
def gill_murray_wright(xk, xkplus1, valk, valkplus1, gradk, k, tau, eps,  
                        kmax):  
    ...  
    return test
```

die die Abbruchkriterien nach Gill, Murray und Wright überprüft. Mit der booleschen Ausgabe-Variable `test` soll die Funktion `True` zurückgeben, wenn eines der drei Kriterien erfüllt ist, und ansonsten `False`. Geben Sie im Falle des Abbruchs vor der Rückgabe in der Konsole aus, welches Kriterium erfüllt ist.

- (iii) Erweitern Sie das bereits implementierte Gradientenabstiegs- und Newtonverfahren um
- die Bestimmung der Schrittweite α mit der Armijo-Backtracking-Liniensuche. Initialisieren Sie die Liniensuche im Gradientenabstiegsverfahren mit dem Maximum aus der für quadratische Funktionen optimalen Schrittweite und 1, und im Newtonverfahren mit $\alpha_0 = 1$.
 - und den Abbruch des Optimierungsverfahrens nach den Abbruchkriterien nach Gill, Murray und Wright.

Schreiben Sie dazu die Funktionen

```
def gradient_descent_erweitert(f, x0, rho=0.5, c1=1e-3, eps=1e-10,  
                               tau=1e-10, kmax=100, xstar=None):
```

und

```
def newton_erweitert(f, x0, rho=0.5, c1=1e-3, eps=1e-10,  
                    tau=1e-10, kmax=100, xstar=None):
```

Sie dürfen bereits implementierte Codeteile aus vorherigen abgegebenen Übungsblättern oder aus deren in Moodle hochgeladenen Lösungsvorschlägen übernehmen und anpassen. Markieren Sie dabei deutlich, welche Codezeilen Sie geändert und/oder hinzugefügt haben.

-
- b) Eine beliebige Testfunktion in der Optimierung ist die Rosenbrock-Funktion $f: \mathbb{R}^2 \rightarrow \mathbb{R}$,

$$f(x, y) = 100(y - x^2)^2 + (1 - x)^2.$$

- (i) Berechnen und klassifizieren Sie die stationären Punkte der Rosenbrock-Funktion von Hand.

- (ii) Implementieren Sie zu f die Python-Funktion

```
def rosenbrock(x, y):  
    ...  
    return val, grad, hess
```

die `val`, `grad` und `hess` ausgewertet an der Stelle $(\mathbf{x}, \mathbf{y})$ zurückgibt. Ihre Funktion soll sowohl für einzelne Vektoren (d.h. skalare $\mathbf{x}$ und $\mathbf{y}$) als auch für n Vektoren gleichzeitig (d.h. $\mathbf{x}$ und $\mathbf{y}$ stellen jeweils $(n,)$ Arrays dar) anwendbar sein. In diesem Fall besitzen `val`, `grad` und `hess` die Dimensionen $(n,)$, $(n, 2)$ bzw. $(n, 2, 2)$.

- (iii) Erstellen Sie einen `surface`-Plot von f für Gitterpunkte auf der Menge $[-3, 3] \times [-3, 3]$.

- c) Testen Sie die beiden modifizierten Verfahren an der Rosenbrock-Funktion für $\mathbf{x}^0 = (0, -0.71)^\top$ und $\mathbf{x}^0 = (-1.2, 1)^\top$ mit den Backtracking-Parametern `rho = 0.5`, `c_1 = 1e-3` und den Abbruchparametern `eps = 1e-10` und $k_{\max} = 1000$ jeweils *einmal für tau = 10⁻² und einmal für tau = 10⁻¹⁰*.

- Messen Sie (z.B. mit `time.time()`) jeweils die gesamte Zeit der Optimierung und geben Sie mit `print` die gerechnete Lösung, die Anzahl benötigter Iterationen sowie die daraus resultierende durchschnittliche Zeit pro Iteration in die Konsole aus.
- Stellen Sie jeweils den Iterationsverlauf $\mathbf{x}^k$ in einem `contour`-Plot sowie die Entwicklung in der Differenz der Zielfunktionswerte $f(\mathbf{x}^k) - f(\mathbf{x}^*)$, der Differenz der Iterierten $\|\mathbf{x}^k - \mathbf{x}^*\|_2$ und der Norm des Gradienten $\|\nabla f(\mathbf{x}^k)\|_2$ dar. Sie dürfen hierzu die in `utils.py` gegebene `plot` Funktion verwenden.

Vergleichen und diskutieren Sie die Ergebnisse.

Hinweis: Der Implementierung entsprechend erwartet die Rosenbrock-Funktion separate Variablen `x`, `y`, während in den Verfahren der Funktion `f` nur eine Variable `x` (hier quasi $\mathbf{x} = (x, y)$). Berücksichtigen Sie dies in der Erstellung von `f`; hilfreich könnte der Python-Operator `*` sein, mit dem Tupel, Listen oder Arrays „aufgelöst“ werden.